

Intro to R - Common Commands (*mosaic*)

B.A. Hartlaub and A.S. Wagaman

Access to R and R Markdown

You'll need access to an installation of the software R and an interface, like RStudio, to work on these assignments. Your institution may have an R or RStudio/Posit server or you may need to do the installations on your own machine.

This assignment is prepared in R Markdown, a markup language that allows you to easily format your code for readability and reproducibility. The benefit of using R Markdown is that it enables you to seamlessly combine R commands, the output from those commands, and written analysis into one document.

In this assignment, we'll explore some common commands and the *mosaic* package, which just contains some useful commands to keep code syntax consistent. If you're not familiar with R, this is a good place to start.

The first step is to load some existing libraries (sets of R commands) that will be helpful. You will get comfortable downloading and installing other R packages as we move throughout the course.

If it's your first time using any particular package (which might occur here), you'll first need to install it. You can install from the Packages menu or use install commands. If you get an error that "there is no package called 'whatever you tried to load'", then you need to install the package first.

Step 1 - Let's load a few libraries. More details about these particular packages are provided in the introduction below.

Introduction

NOTE: R is a programming language - there are often multiple ways to accomplish a given task. You can choose to work with any set of commands that works. Use your class notes and examples too! We are using a mix of `ggformula` and `ggplot2` for plots, so that may be different from what you have seen before. Other options exist.

We often begin by loading the **`mosaic`** and *NSM3* package. *mosaic* is a useful add-on for R and *NSM3* is the package for Hollander, Wolfe, and Chicken's (HWC) Nonparametric textbook. Make sure to include the `include = FALSE` option in the chunk that loads **`mosaic`** and other packages. This will suppress unwanted messages and keep your Markdown document neat and clean. Above, you can see how this is done. We've also loaded another package that we use below.

Note that R is case-sensitive. That means capitalization and spelling count! The `#` in a line of code is a commenting convention. R ignores anything after the `#` as a comment, but it will still show up in the line of code. There are other ways to make code not show up that you can learn about.

The lines below under *calculations* make up a **code chunk**, that is, a block of R code embedded into the RMarkdown file. Once you click on **Knit PDF** (or a different Knit option) at the top of the RStudio interface, you'll see the generated file which not only contains code from your code chunk, but also the processed R output. However, you usually only want to knit when you are ready to get a full version of the document, not just as you are working.

You can get the output as you go in the console window or in line (based on the options you have set) by running each chunk without knitting. Just click the run button while in a chunk or on a line, or use the

green arrow or the keyboard shortcuts (Ctrl+Enter for a single line, etc. (check out the run menu)). For the sections below, try running with the green arrow in each chunk, or line by line. Be sure to run the code chunk above that sets up all the packages before proceeding below!

Calculations

R is first and foremost a (scientific) calculator. Let's try a few things:

```
3 + 15
```

```
## [1] 18
```

```
4 + 5*6
```

```
## [1] 34
```

```
(8+5)*2
```

```
## [1] 26
```

```
3^2
```

```
## [1] 9
```

Note that R doesn't understand multiplication without an asterisk. Typing 2(5) will give an error, not the value of 10.

Creating variables

Not only can we calculate things, we can also store values (like our calculations). To do this, we use the assignment <- operator for assignment.

```
x <- 16 # this creates a variable `x` with the value 16 stored
```

```
x # if we just type `x` without an assignment, we can now see what's stored in `x`
```

```
## [1] 16
```

```
# y # would give an error (i.e., Error: object 'y' not found) because there is no "y" yet
```

It's often useful to create a variable that saves the output of a calculation:

```
y <- 4 + 5 * 6
```

Mostly, you'll see this operator used to store data sets and certain analysis results, which we'll explore later.

Reading in and Working with Data

Let's start by loading some data. We can either load data that already exists in R using the data function or read in data sets that are provided elsewhere.

For this exploration of commands, we'll use the *Pines* dataset from the Stat2 textbook loaded within R via the Stat2Data package. You can learn about this data set from the help menu.

Remember, the code below will run if you click Run from your menu, or put your cursor in it and hit Ctrl+Alt+C or hit Run Current Chunk from the Chunks menu. Remember to run the chunks as you work, or no output will appear!

```
data(Pines)
```

Pines is now in our workspace (called the Environment pane) as a *data.frame*. Try clicking on the name of the data set in the Environment pane. This lets you learn a little about the data set. The help menu will define the variables. You can get this by running ?Pines or help(Pines) as commands in the Help pane or

Console pane, because the name of the dataset is Pines. (Not all data sets will have help menus, but those from R packages will.)

What else can we learn about the **Pines** data set?

```
# Some useful R commands - Remember to RUN this code chunk
```

```
# What are its dimensions?
```

```
dim(Pines)
```

```
## [1] 1000 15
```

```
# What variables are present?
```

```
names(Pines)
```

```
## [1] "Row" "Col" "Hgt90" "Hgt96" "Diam96" "Grow96"
```

```
## [7] "Hgt97" "Diam97" "Spread.97" "Needles97" "Deer95" "Deer97"
```

```
## [13] "Cover95" "Fert" "Spacing"
```

```
# A brief summary of each variable
```

```
summary(Pines)
```

```
##      Row      Col      Hgt90      Hgt96
## Min.   : 1.00  Min.   : 1.00  Min.   : 6.00  Min.   : 11.0
## 1st Qu.:14.00  1st Qu.: 6.00  1st Qu.:16.00  1st Qu.:235.0
## Median :26.00  Median :12.00  Median :19.00  Median :289.0
## Mean   :24.76  Mean   :12.41  Mean   :19.20  Mean   :279.3
## 3rd Qu.:35.00  3rd Qu.:18.00  3rd Qu.:22.38  3rd Qu.:329.0
## Max.   :44.00  Max.   :30.00  Max.   :39.00  Max.   :491.0
##                                     NA's   :182   NA's   :139
##      Diam96      Grow96      Hgt97      Diam97
## Min.   :0.500  Min.   : 0.00  Min.   : 87.0  Min.   : 0.400
## 1st Qu.:3.050  1st Qu.: 64.00  1st Qu.:301.0  1st Qu.: 4.700
## Median :4.300  Median : 78.00  Median :357.0  Median : 6.200
## Mean   :4.103  Mean   : 74.17  Mean   :346.6  Mean   : 5.942
## 3rd Qu.:5.300  3rd Qu.: 90.00  3rd Qu.:406.0  3rd Qu.: 7.400
## Max.   :8.100  Max.   :126.00  Max.   :558.0  Max.   :10.700
## NA's   :149   NA's   :136   NA's   :135   NA's   :139
##      Spread.97      Needles97      Deer95      Deer97
## Min.   : 20.0  Min.   : 25.00  Min.   :0.00  Min.   :0.0000
## 1st Qu.:151.8  1st Qu.: 65.00  1st Qu.:0.00  1st Qu.:0.0000
## Median :185.0  Median : 74.00  Median :0.00  Median :0.0000
## Mean   :182.6  Mean   : 73.85  Mean   :0.24  Mean   :0.1064
## 3rd Qu.:214.2  3rd Qu.: 82.00  3rd Qu.:0.00  3rd Qu.:0.0000
## Max.   :339.0  Max.   :172.00  Max.   :1.00  Max.   :1.0000
## NA's   :136   NA's   :135   NA's   :129   NA's   :135
##      Cover95      Fert      Spacing
## Min.   :0.000  Min.   :0.000  Min.   :10.00
## 1st Qu.:0.000  1st Qu.:0.000  1st Qu.:10.00
## Median :1.000  Median :1.000  Median :10.00
## Mean   :1.328  Mean   :0.507  Mean   :12.07
## 3rd Qu.:2.000  3rd Qu.:1.000  3rd Qu.:15.00
## Max.   :3.000  Max.   :1.000  Max.   :15.00
##
```

```
# A glimpse of the data set; no summary info but has variable types
```

```
glimpse(Pines)
```

```
## Rows: 1,000
```

```
## Columns: 15
## $ Row      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, ~
## $ Col      <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 1~
## $ Hgt90    <dbl> NA, 14.0, 17.0, NA, 24.0, 22.0, 18.0, 32.0, NA, 22.0, 17.0, ~
## $ Hgt96    <dbl> NA, 284.0, 387.0, NA, 294.0, 310.0, 318.0, 328.0, 157.0, 282~
## $ Diam96   <dbl> NA, 4.2, 7.4, NA, 3.9, 5.6, 5.4, 5.4, 1.3, 4.5, 3.3, 3.4, 3.~
## $ Grow96   <dbl> NA, 96.0, 110.0, NA, 70.0, 84.0, 96.0, 88.0, 64.0, 83.0, 71.~
## $ Hgt97    <int> NA, 362, 442, NA, 369, 365, 356, 365, 208, 329, 298, 245, 31~
## $ Diam97   <dbl> NA, 6.60, 9.30, NA, 7.00, 6.90, 7.60, 7.70, 2.00, 6.10, 3.90~
## $ Spread.97 <int> NA, 162, 250, NA, 176, 215, 238, 219, 127, 209, 154, 167, 18~
## $ Needles97 <dbl> NA, 66.0, 77.0, NA, 72.0, 76.0, 74.5, 60.5, 56.0, 79.5, 70.5~
## $ Deer95   <int> NA, 0, 0, NA, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, NA, 1, NA,~
## $ Deer97   <int> NA, 1, 0, NA, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, NA, 0, NA,~
## $ Cover95  <int> 0, 2, 1, 0, 2, 1, 0, 1, 2, 2, 1, 0, 2, 1, 2, 2, 0, 2, 0, 3, ~
## $ Fert     <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, ~
## $ Spacing  <int> 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15, ~
```

```
# Another way, Similar to glimpse, to look at the structure of a data set
str(Pines)
```

```
## 'data.frame':    1000 obs. of  15 variables:
## $ Row      : int  1 1 1 1 1 1 1 1 1 1 1 ...
## $ Col      : int  1 2 3 4 5 6 7 8 9 10 ...
## $ Hgt90    : num  NA 14 17 NA 24 22 18 32 NA 22 ...
## $ Hgt96    : num  NA 284 387 NA 294 310 318 328 157 282 ...
## $ Diam96   : num  NA 4.2 7.4 NA 3.9 5.6 5.4 5.4 1.3 4.5 ...
## $ Grow96   : num  NA 96 110 NA 70 84 96 88 64 83 ...
## $ Hgt97    : int  NA 362 442 NA 369 365 356 365 208 329 ...
## $ Diam97   : num  NA 6.6 9.3 NA 7 6.9 7.6 7.7 2 6.1 ...
## $ Spread.97: int  NA 162 250 NA 176 215 238 219 127 209 ...
## $ Needles97: num  NA 66 77 NA 72 76 74.5 60.5 56 79.5 ...
## $ Deer95   : int  NA 0 0 NA 0 0 0 0 1 0 ...
## $ Deer97   : int  NA 1 0 NA 0 0 0 0 1 1 ...
## $ Cover95  : int  0 2 1 0 2 1 0 1 2 2 ...
## $ Fert     : int  0 0 0 0 0 0 0 0 0 1 ...
## $ Spacing  : int  15 15 15 15 15 15 15 15 15 15 ...
```

```
# The first six rows of the data set
head(Pines)
```

```
##   Row Col Hgt90 Hgt96 Diam96 Grow96 Hgt97 Diam97 Spread.97 Needles97 Deer95
## 1   1   1   NA    NA     NA     NA     NA     NA         NA         NA    NA
## 2   1   2   14   284    4.2    96    362    6.6        162         66     0
## 3   1   3   17   387    7.4   110    442    9.3        250         77     0
## 4   1   4   NA    NA     NA     NA     NA     NA         NA         NA    NA
## 5   1   5   24   294    3.9    70    369    7.0        176         72     0
## 6   1   6   22   310    5.6    84    365    6.9        215         76     0
##   Deer97 Cover95 Fert Spacing
## 1     NA      0    0      15
## 2      1      2    0      15
## 3      0      1    0      15
## 4     NA      0    0      15
## 5      0      2    0      15
## 6      0      1    0      15
```

```
# The last three rows of the data set
tail(Pines, 3)
```

```
##      Row Col Hgt90 Hgt96 Diam96 Grow96 Hgt97 Diam97 Spread.97 Needles97 Deer95
## 998   44  25   11   319    4.2    91   381    7.2      241      63      1
## 999   44  26   19   231    2.4    76   263    4.9      127      63      1
## 1000  44  27    NA    NA     NA     NA    NA     NA      NA      NA     NA
##      Deer97 Cover95 Fert Spacing
## 998      1      0    0      10
## 999      1      0    0      10
## 1000    NA      0    0      10
```

Univariate Analysis

Often, we'll want to explore basic graphs and descriptive statistics for variables in the data set. Here are examples of some common commands.

We will use the *formula* interface whenever possible - **mosaic** helps convert many other R functions to have this format, helping us have syntax consistency. This means using the $\sim$ (tilde) operator. The general format of these commands is:

```
commandname(Y ~ X, data = datasetname)
```

Some commands only have a single variable, i.e. only the X. When learning a new command, make sure you understand the various components. You can run `?commandname` or `help(commandname)` in the console pane to open the help menu for the command.

Let's look at the Hgt90 variable in Pines. This is the tree height at time of planting in 1990.

To get a basic summary of a quantitative variable, use *favstats()*.

```
# descriptive stats of Hgt90 variable in Pines data set
favstats(~ Hgt90, data = Pines)
```

```
##  min Q1 median      Q3 max      mean      sd  n missing
##    6 16     19 22.375  39 19.20061 5.634271 818      182
```

To specifically retrieve the mean or standard deviation, there are specific commands you can use, but *favstats* gives them as well.

```
mean(~ Hgt90, data = Pines, na.rm = TRUE)
```

```
## [1] 19.20061
```

```
sd(~ Hgt90, data = Pines, na.rm = TRUE)
```

```
## [1] 5.634271
```

Note: We have added the option `na.rm = TRUE` to remove NAs, which are missing observations. Without this option, you would get NA for these values. Not all functions have defaults to deal with missing observations.

You can store these values to new variables if you want to use them later on. When you execute the code chunk below, you will see "mu" pop up as a new value in the environment pane.

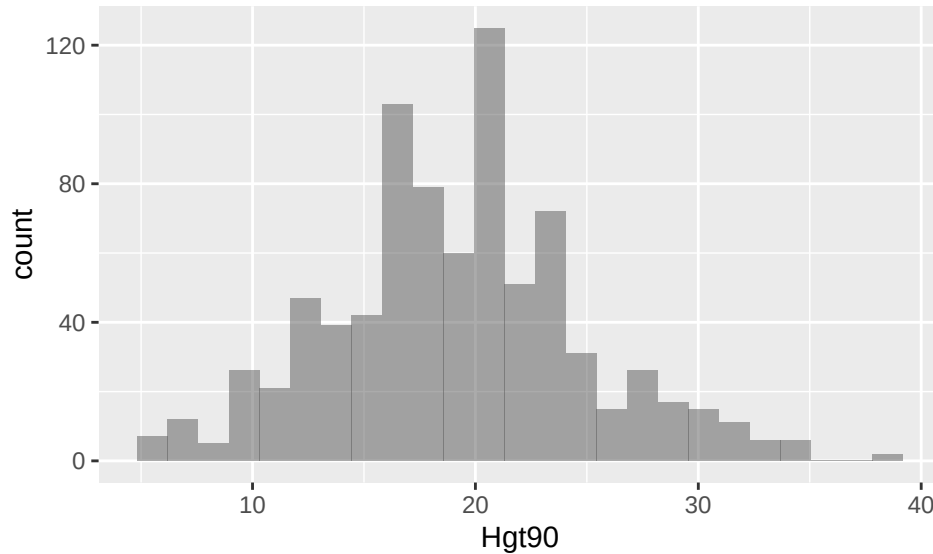
```
mu <- mean(~ Hgt90, data = Pines, na.rm = TRUE)
mu
```

```
## [1] 19.20061
```

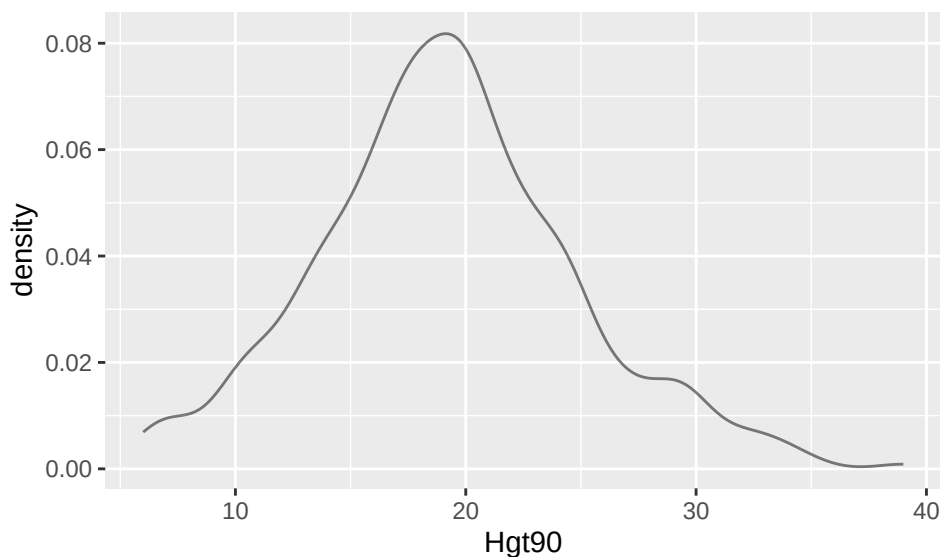
For a graphical description of the distribution of a single variable, we can create a histogram or a density plot. Once you generate plots, you can cycle through them using the arrow keys in the Output pane. (Note:

If the plots tab is not selected, you may have to click the plots tab first. You can also use the arrows in the bar below the output pane options.)

```
# gf_dhistogram will give the density version of a histogram; default is counts
gf_histogram(~ Hgt90, data = Pines)
```



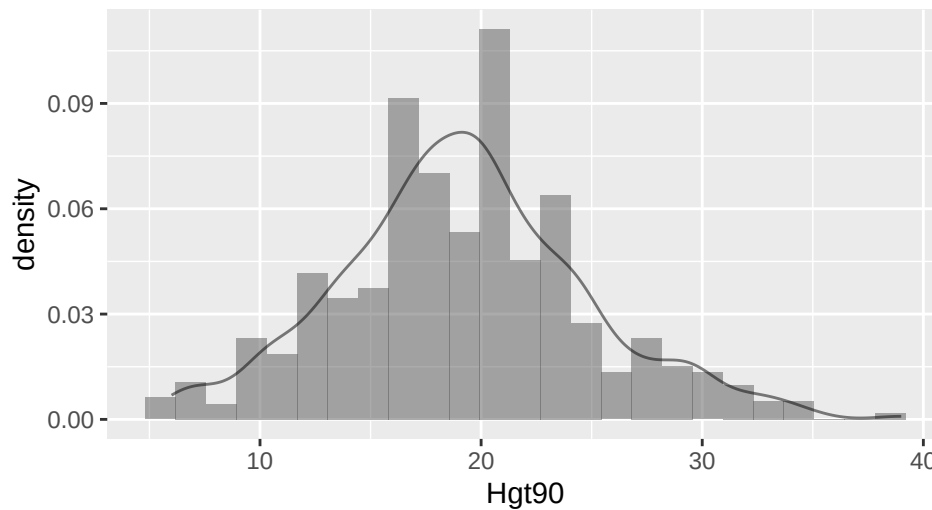
```
gf_dens(~ Hgt90, data = Pines)
```



The `gf_` prefix is for our usage of `ggformula`. We can do lots of neat things with our plots, such as overlay them and give them titles. Here is an example. Note that we use the piping operator `%>%`, which passes the output from one command into another. There is a newer piping operator `/>` that you can also use.

```
gf_dhistogram(~ Hgt90, data = Pines) %>% #note swap to gf_dhistogram for scaling
  gf_dens() %>%
  gf_labs(title = "Overlaid Histogram and Densityplot for Hgt90 Values")
```

Overlaid Histogram and Densityplot for Hgt90 Values



We'll explore ggplot2 and more plotting ideas in our separate Intro to R - Ggplot2 file.

Don't worry about learning all these commands right away. Use the examples to help you! Focus on learning what the sets of commands do, generally speaking, then when you want to do something similar (but say, on a different data set), copy and paste the code you have as an example and edit the variables and data set to your situation.

If the variable you are interested in is a categorical variable, you might try a table of counts via *tally* or a bar plot as part of your univariate analysis.

Our **Pines** data set has some categorical variables but all are coded numerically. Briefly, we show how to set one as a categorical variable to demonstrate univariate analysis.

The *Fert* and *Spacing* variables are binary variables representing whether fertilizer was used and the spacing between trees. We use the *mutate* command to make them factors (so R understands they are categorical).

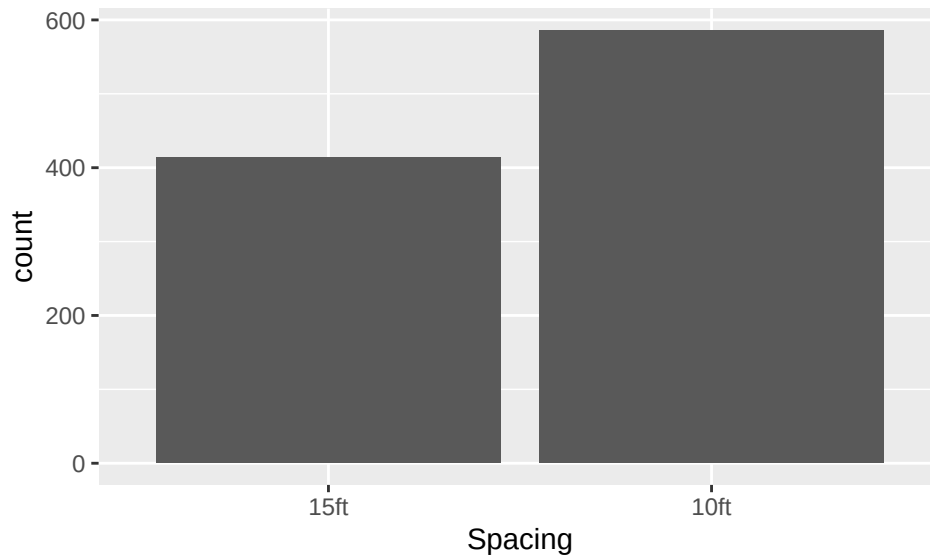
```
Pines <- mutate(Pines,
  Fert = factor(Fert, levels = c(0, 1),
    labels = c("No", "Yes")),
  Spacing = factor(Spacing, levels = c(15, 10),
    labels = c("15ft", "10ft")))
```

Now, let's do a univariate examination of the *Spacing* variable.

```
tally(~ Spacing, data = Pines)
```

```
## Spacing
## 15ft 10ft
##  414  586
```

```
gf_bar(~ Spacing, data = Pines)
```



The table conveys the information in a more compact fashion than the plot.

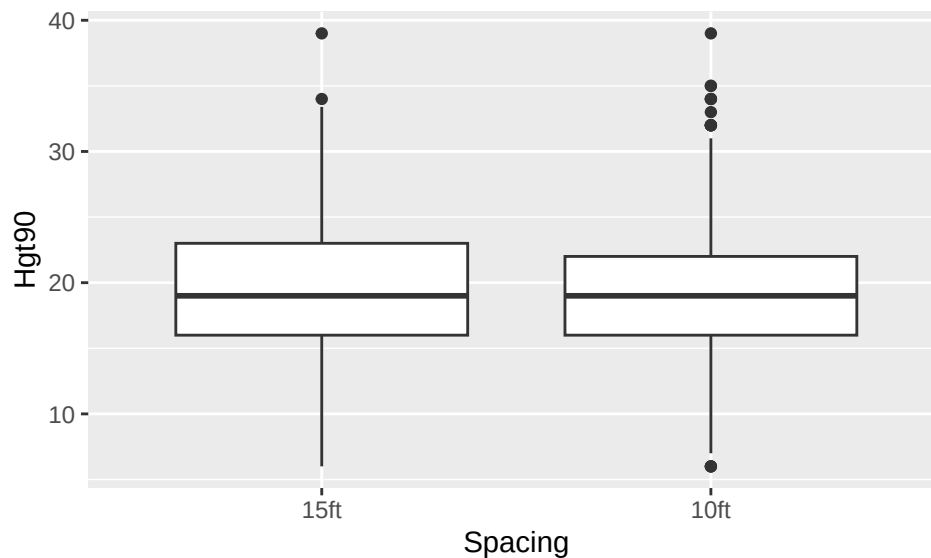
Bivariate Analysis

Often, we examine more than one variable at a time to investigate their relationships. In that case, a variety of plots may be useful, depending on the types of variables involved.

For relating a quantitative variable to a categorical variable, we can draw side-by-side boxplots. This plot, for example, allows us to look at the height of the pine trees at planting and if there were differences based on spacing. (You should note that since is the start of an experiment, we don't want to see differences here.)

```
gf_boxplot(Hgt90 ~ Spacing, orientation = "x", data = Pines)
```

```
## Warning: Removed 182 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Descriptive statistics by group can also be useful here. Note the quantitative variable goes before the tilde, and the categorical variable goes after.

```
favstats(Hgt90 ~ Spacing, data = Pines)
```

```
##   Spacing min Q1 median Q3 max   mean   sd  n missing
## 1   15ft   6 16   19 23  39 19.31281 5.731915 320     94
## 2   10ft   6 16   19 22  39 19.12851 5.575232 498     88
```

What if both variables involved are categorical? We can make a contingency table using tally, or you can explore potential plots (may require other packages).

```
tally(~ Spacing + Fert, data = Pines, margins = TRUE)
```

```
##           Fert
## Spacing  No  Yes Total
##   15ft   208 206  414
##   10ft   285 301  586
##   Total  493 507 1000
```

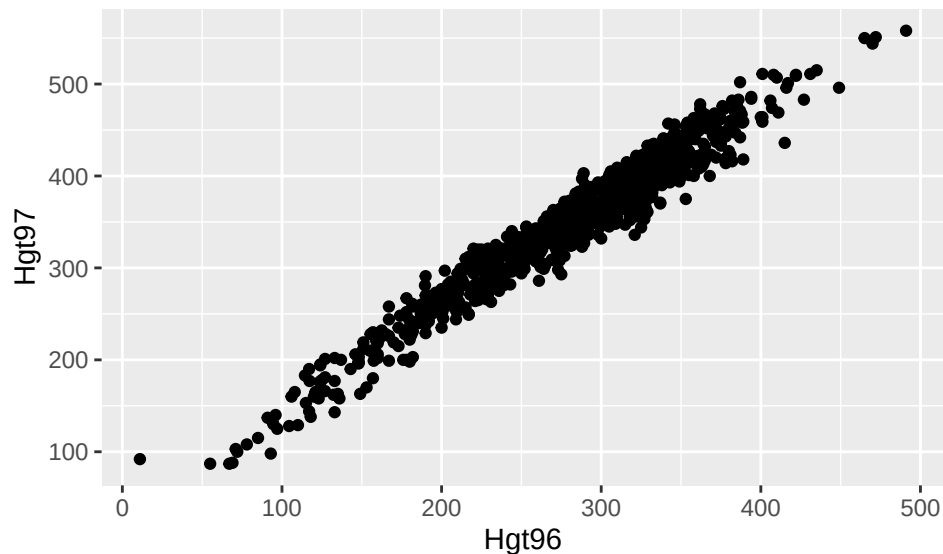
```
tally(~ Spacing + Fert, data = Pines, margins = TRUE, format = "percent")
```

```
##           Fert
## Spacing  No   Yes Total
##   15ft   20.8 20.6  41.4
##   10ft   28.5 30.1  58.6
##   Total  49.3 50.7 100.0
```

Again, considering the context, Fert and Spacing should be independent. Does the analysis support this?

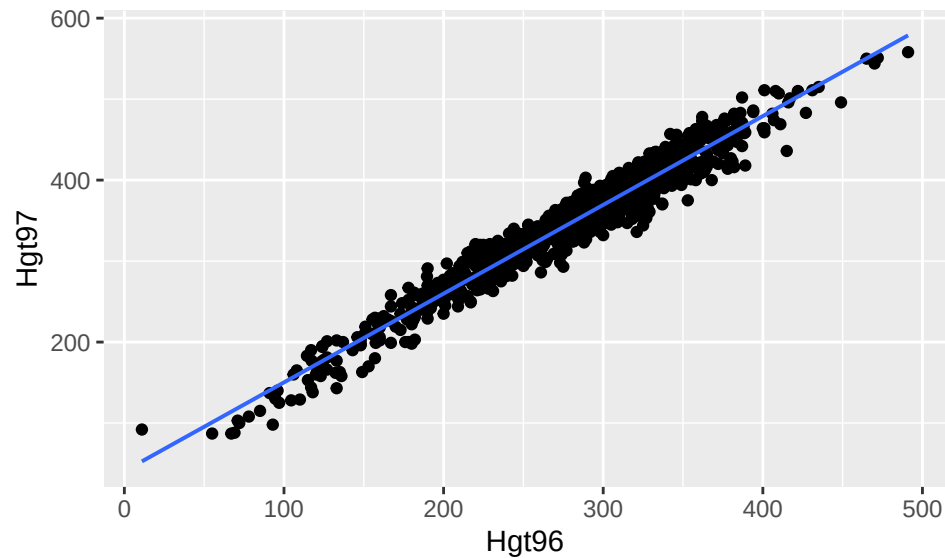
Finally, if the two variables involved are quantitative, we typically make a scatterplot. If appropriate, you can compute correlation and fit a linear regression line. Here, we might naturally assume there is a relationship between Hgt96 and Hgt97, so let's examine those a bit more closely. Note that we put Hgt96 as the X variable, so it goes after the tilde, while the response, Hgt97, goes in front.

```
gf_point(Hgt97 ~ Hgt96, data = Pines)
```



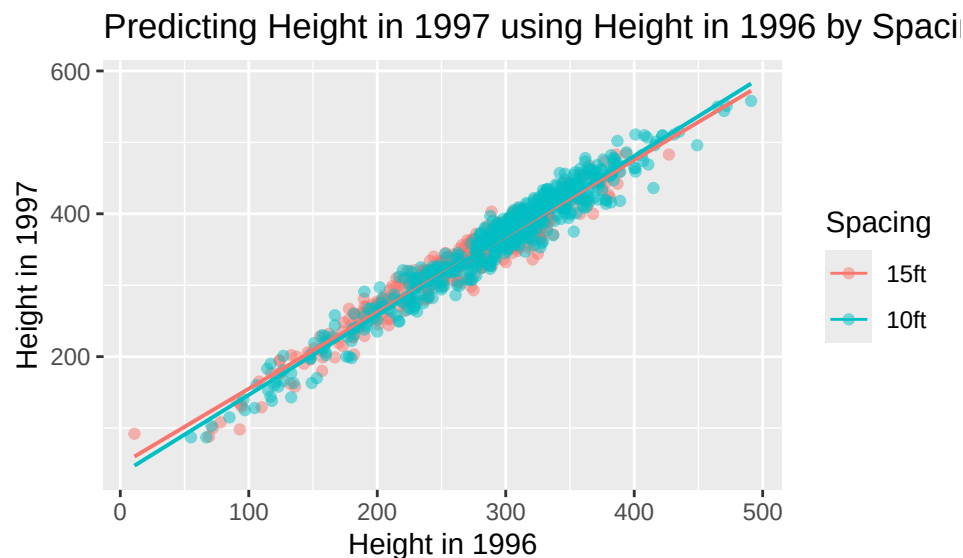
This shows a strong positive linear relationship, so we can add the regression line to this plot.

```
gf_point(Hgt97 ~ Hgt96, data = Pines) %>%
  gf_lm()
```



Note that there are many graphical options that we can add to this plot - some of which add more variables. Here is the same basic plot, but jazzed up a bit. Feel free to experiment with graphical options! If you don't know what is available, hit the tab key inside the function call. Or use the Help menu options. Remember to use examples to help you get code that works for what you want.

```
gf_point(Hgt97 ~ Hgt96, data = Pines, color = ~ Spacing, alpha = 0.5,
         xlab = "Height in 1996", ylab = "Height in 1997") %>%
  gf_lm() %>% #adds linear regression line
  gf_labs(title = "Predicting Height in 1997 using Height in 1996 by Spacing")
```



Note that in this case we have conditioned on a third variable **Spacing**, via the color argument.

For a numerical assessment of the linear relationship between two quantitative variables, we can compute their correlation coefficient.

```
cor(Hgt97 ~ Hgt96, data = Pines)
```

```
## [1] NA
```

NA? What? Sometimes the `cor` function will not work due to missing data in the data frame. When that happens, you can still get the correlation between all pairs of observations without missing data like this:

```
cor(Hgt97 ~ Hgt96, data = Pines, use = "pairwise.complete.obs")
```

```
## [1] 0.973998
```

Based on your work so far, a simple linear regression model is reasonable, so let's look at fitting the model and getting plots to check our conditions as our last bit of code.

```
fm <- lm(Hgt97 ~ Hgt96, data = Pines)
msummary(fm) #need the summary
```

```
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 40.590784   2.524484   16.08  <2e-16 ***
## Hgt96       1.096059   0.008734   125.49  <2e-16 ***
##
## Residual standard error: 18.47 on 852 degrees of freedom
## (146 observations deleted due to missingness)
## Multiple R-squared:  0.9487, Adjusted R-squared:  0.9486
## F-statistic: 1.575e+04 on 1 and 852 DF,  p-value: < 2.2e-16
```

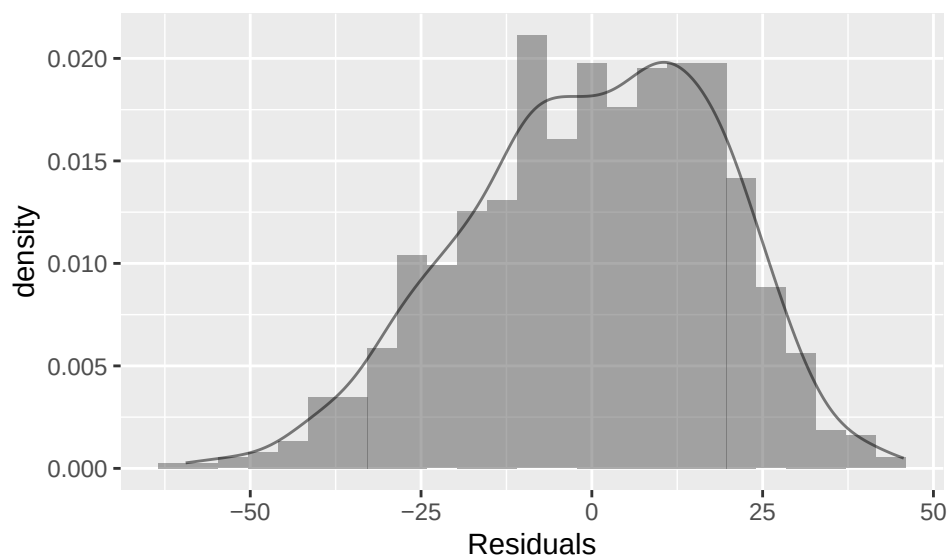
Note that *fm* stands for fitted model, but you can use any name you want for the model. We use the assignment operator to “save” the model because we call it in several later commands. Common names for models you might see in class are things like *mod*, *mod1*, *mymod*, etc. Just be sure to keep track of what name you are using for calling the summaries and other related commands.

From the output, what is the equation of the fitted regression line?

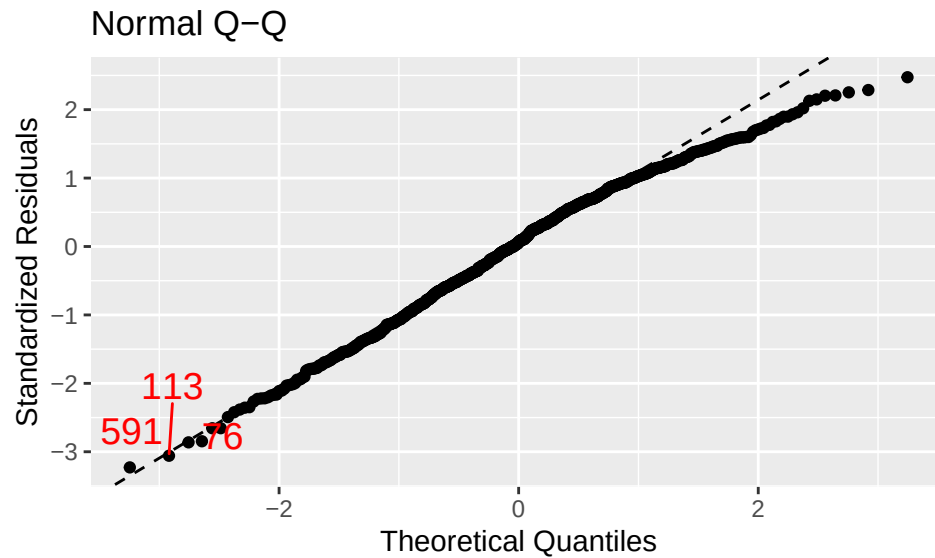
How do we check conditions? There are several ways to get the necessary plots. Some are easier than others.

Normality of errors condition (2 ways):

```
# generates a histogram with fitted density curve
gf_dhistogram(~ residuals(fm), xlab = "Residuals") %>%
  gf_dens()
```



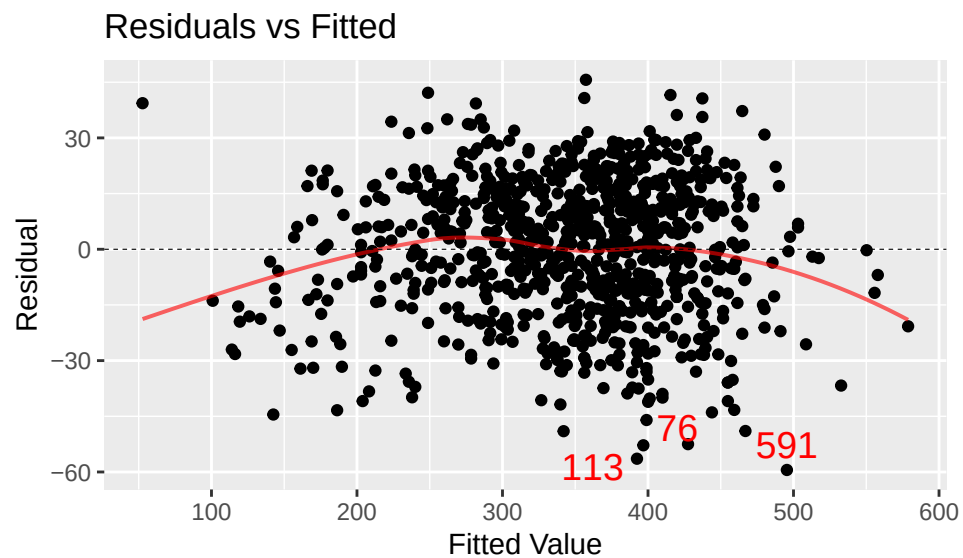
```
# generates the associated qqplot
mplot(fm, which = 2)
```



Constant Variance condition:

```
# Residuals vs. fitted plot
mplot(fm, which = 1)
```

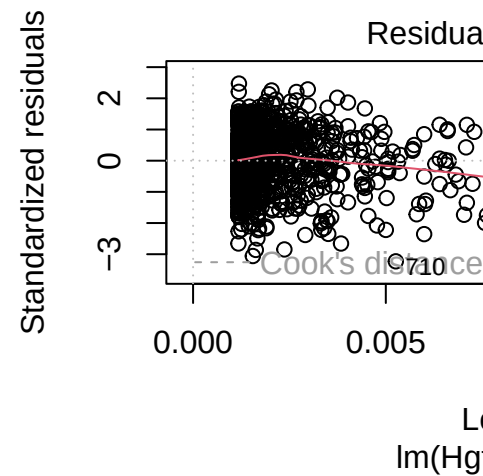
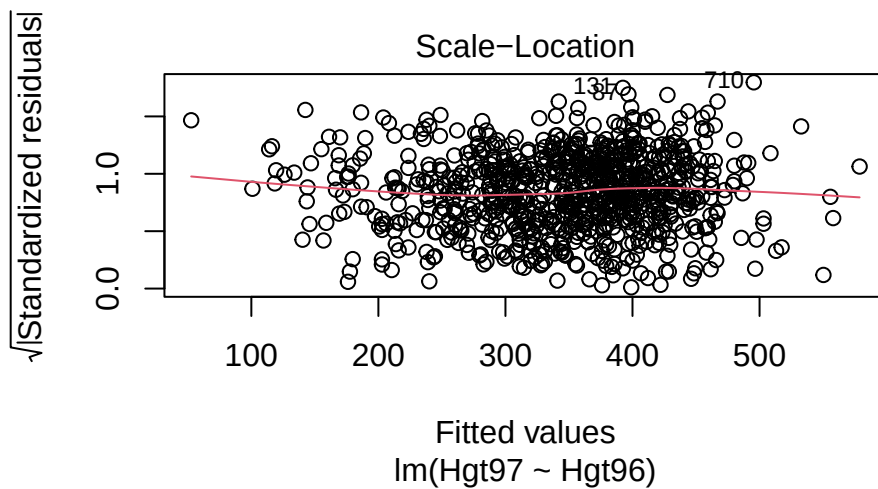
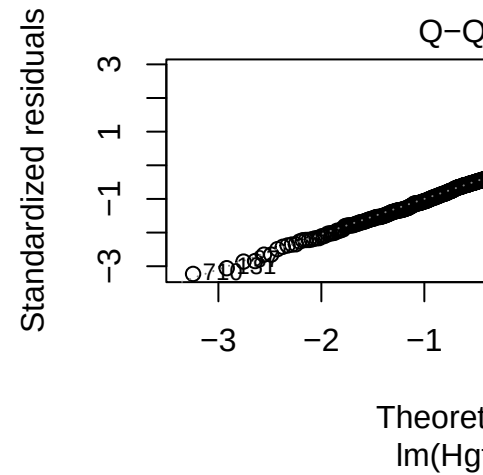
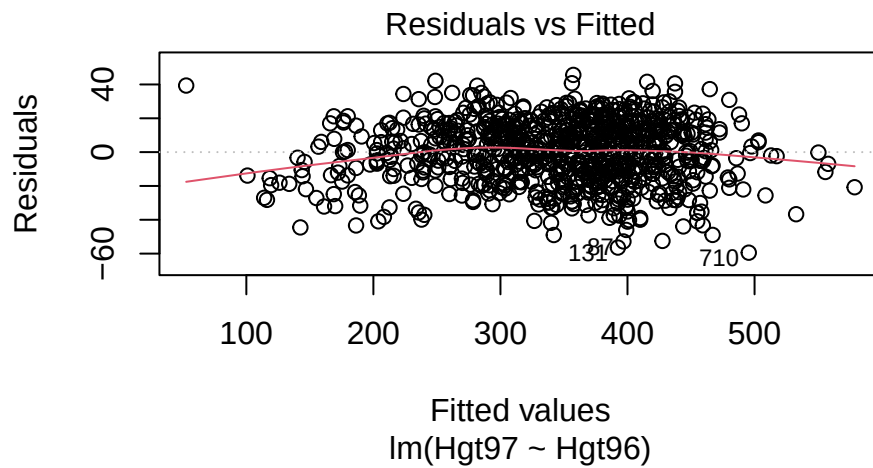
```
## `geom_smooth()` using formula = 'y ~ x'
```



What do you think about the conditions necessary for simple linear regression (SLR) to be appropriate?

Another way to assess the conditions of the model is to create several plots at one time with the plot command and the object from your fitted model.

```
plot(fm)
```



Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to Nonparametric Statistical Methods (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.